

课程笔记

Adaptive Architecture for Pentest Agents: 绿盟科技 AI 小分队的智能渗透 Harness 设计

讲者：绿盟科技 AI 小分队 笔记整理：五道口纳什 & Codex

2026 年 5 月 3 日



视频作者/频道：腾讯安全众测（B 站）

发布日期：2026-05-02

视频时长：15:55

视频链接：<https://www.bilibili.com/video/BV1PQ9YBKEL6/>

比赛背景：第二届腾讯云黑客松（TCH）智能渗透挑战赛·决赛

演讲主题：Adaptive Architecture for Pentest Agents —Manager, Observer, Solver

目录

1 引言与四条设计理念	3
1.1 从第 7 名到第 1 名：背景与动机	3
1.2 基线扫视：四个开源 Pentest Agent	3
1.3 四条设计理念：整篇讲座的“宪法”	4
1.4 本章小结	5
2 整体架构与工程实践	7
2.1 基础框架：PydanticAI (PydanticAI SDK)	7
2.2 控制面 / 执行面解耦	7
2.3 Kali 容器隔离：为什么必须容器化	9
2.4 消息总线与统一状态回传	9
2.5 工程实践四要素：从架构到产品	9
2.6 本章小结	10
3 Manager Agent：调度与边际效益最大化	12
3.1 比赛硬约束：三个并发卡槽	12
3.2 从战场感知到动态规划	12
3.3 LLM 做规划，工程做边界	13
3.4 Manager Tools 八件套	13
3.5 Strategy Actions 六件套：六类策略输出	15
3.6 本章小结	15
4 Solver Agent：并行探索与共享状态	16
4.1 第一性原则：探索不应消失	16
4.2 并行探索的隔离设计	16
4.3 Observer Sidecar：旁观者沉淀	16
4.4 两层 Memory：局部 vs 全局	17
4.5 广播机制：信号 → 消息总线 → 所有 Solver	18
4.6 本章小结	19
5 Harness 四大创新：Solver Harness 与 Context Harness	20
5.1 Harness 框架：稳定运行的四个支柱	20
5.2 创新一：智能纠偏 (Efficiency Reminder)	20
5.2.1 痛点：长任务里的“漂移”	20
5.2.2 机制：Observer 当裁判，但不当主角	21
5.3 创新二：Memory & Idea 双层状态板	22
5.3.1 真正失去的不是能力，而是状态本身	22
5.3.2 Idea Board：策略层的状态机	22
5.3.3 Memory Board：事实层的四类槽位	22
5.4 创新三：Ralph-Loop (终止控制环路)	23
5.4.1 字幕里的“REFLOOP”是错听	23
5.4.2 为什么“该不该结束”不能交给模型自己判断	23

5.4.3	终止由”外部条件”接管	24
5.5	创新四：Context Harness 之 Compaction 四步流水线	25
5.5.1	上下文腐败的两大来源	25
5.5.2	第 1 步——Raw CLI Output（原始命令行输出）	25
5.5.3	第 2 步——RTK Rewrite（工具结果重写）	25
5.5.4	第 3 步——Context Compression / Compaction（上下文压缩）	25
5.5.5	第 4 步——Solver Context（注入 Solver 的紧凑上下文）	26
5.5.6	Progressive Disclosure: Skill 系统的同源思想	26
5.6	本章小结：四大创新如何共同构成”稳定运行环境”	26
6	实验数据、Q&A 与总结	28
6.1	实验数据一：成本与一血/二血/三血分布	28
6.2	实验数据二：海外 + 国产模型并行的 Model Performance	29
6.3	Q&A 第一轮：与 Claude Code 范式的差异	30
6.3.1	Q1：三个亮点要”不依赖模型，也不依赖 Claude Code”	30
6.3.2	Q2：推理框架是 Claude Code/Codex，还是自研？	30
6.3.3	Q3：能否量化证明优于 Claude Code？	31
6.4	Q&A 第二轮：Idea 维护与上下文压缩	31
6.4.1	Q：Idea 怎么保证不被改写、攻击方向不偏离？	31
6.4.2	Q：上下文压缩具体怎么做？	32
6.5	Q&A 第三轮：全局 Memory 增长可控的三道闸门	32
6.6	总结与延伸	33
6.6.1	6.1 团队主张的核心贡献	33
6.6.2	6.2 与 Claude Code 范式的关系	33
6.6.3	6.3 笔者的结构化提炼	34
6.6.4	6.4 未来工作与开放问题	34
6.6.5	6.5 一句话收尾	35

1 引言与四条设计理念

1.1 从第 7 名到第 1 名：背景与动机

绿盟科技 AI 小分队(NSFOCUS AI Squad, 舞台 Logo 为“EZ”)参加的是腾讯云黑客松(Tencent Cloud Hackathon, TCH) 第二届智能渗透挑战赛决赛。值得玩味的是, 团队在第一届赛事上仅获第 7 名, 而本届一举夺冠。两届之间间隔的几个月中, 他们并没有等到一个划时代的新模型, 相反, 模型与去年大体同源; 真正变化的, 是围绕模型的那一层工程脚手架。换句话说, 本场分享给出的核心命题是:

当

LLM 自身的能力大致给定时, 决定渗透测试 Agent (Pentest Agent) 整体战斗力的, 不是再换一个更强的模型, 而是围绕模型构建的 **Harness** 与协同架构——也就是后文反复出现的“Adaptive Architecture”。

围绕这个命题, 演讲沿着幻灯片 PART 01 / PART 02 / PART 03 的顺序, 分三部分展开:

1. **整体架构 (Overall Architecture):** 从控制面与执行面的解耦讲到工程平台化能力, 本笔记对应第 2 节。
2. **Harness 设计 (Harness Design):** 聚焦 Solver 与 Observer Sidecar 上的四项关键创新 (智能纠偏、Memory & Idea、Ralph-Loop、上下文压缩), 对应第 3 至第 5 节。
3. **总结与 Q&A:** 实测数据、与评委的三轮问答, 以及对 Claude Code 范式的对比反思, 对应第 6 节。

讲者特别强调, 这一年的“打磨”并非在某一个炫技点上发力, 而是先把一组开源 Pentest Agent 系统拉出来横向对比, 从中抽象出共通的设计公约数; 再以此为基础重写整套架构。也就是说, “四条设计理念”并非凭空设定, 而是被开源生态逼出来的工程经验。

1.2 基线扫视：四个开源 Pentest Agent

为了避免“重新发明轮子”, 团队首先扫视了当前社区中具有代表性的四个开源 Pentest Agent 项目。它们各自的工程取向并不一致, 但在某些原则上呈现出相当强的趋同性, 恰好是设计理念的来源。

社区横向对比的四个开源 Pentest Agent

- **Keygraph SHANNON**: 定位为 “AI Pentester for Web Apps and APIs”，主打面向 Web 与 API 的自动化攻击。
- **PentestAgent**: 以 “Pentest Agent” 直接命名的开源项目，工程上较早探索任务规划与工具调用拼装。
- **PentAGI**: “AI agents for penetration testing”，强调多 Agent 协作与任务分解。
- **CAI (Cybersecurity AI)**: “Bug bounty-ready AI”，更偏向赏金猎人场景下的实战流水线。

读者不必逐一精读它们，只需把它们视作同一坐标系中的四个邻居：每个项目都在不同程度上回答了“如何把 LLM 嵌入渗透流程”这一共性问题。

讲者总结道：“呃他们的工程设计上是各有不同，但是在那个设计的相同性上，是有一定的那个相同性。”——也就是说，这四个项目在表层架构（任务图、Agent 拓扑、工具集合）上五花八门，但在更深一层的“结构性约束”上几乎口径一致。把这些口径一致的部分提炼出来，正是 NSFOCUS 团队的第一步抽象。

1.3 四条设计理念：整篇讲座的“宪法”

经过对开源生态的扫视和上届比赛的复盘，团队抽象出四条贯穿全场分享的“宪法”。每一条都对应一个具体的工程痛点，并且每一条都会在后继模块中找到落点。我们先逐条讲清楚动机、机制和后果，再把它们打包成一张简短的 importantbox 以便随时查阅。

原则一：智能决策要有工程边界。 LLM 擅长在不确定的态势中作判断，但天然不擅长遵守资源、时序、安全这些“硬性边界”。如果让模型直接 control 一切，它会写脚本、调工具、起进程，最终出现资源不可控、行为不可审计的问题。因此“智能决策”必须用一层“工程边界”包住——LLM 负责做什么，工程负责允许做什么、什么时候停。这条原则后续会在 Manager Agent 的 Harness/Policy Constraints 中得到最直白的落地。

原则二：执行环境必须隔离可重置。 渗透测试 Agent 的本质工作是“跑攻击载荷”。它会执行命令行工具、写入临时文件、安装新的 payload、与可疑目标通信。如果直接跑在宿主机上，轻则“本机环境污染”（残留工具、服务、依赖冲突），重则被靶机反制。**容器化隔离 + 可重置**是工程上的最低要求。这条原则在第 2 节会以 Kali 容器执行平面的形式落地。

原则三：经验要沉淀成共享状态。 讲者反复提到一句朴素的话——“我们的每一个 token 都是费钱的，不要让它浪费掉”。如果每个 Solver 都从空白上下文开始，相同的失败边界会被多个 Agent 反复踩；相同的有效线索也会被多个 Agent 重新发现。“经验沉淀”意味着把已经被验证的事实、被走通的路径、被排除的方向，写入一个跨 Agent 的共享存储里，新的 Agent 启动时就先继承这些经验再开工。这条原则在第 4 节将以 Challenge Memory / Challenge Idea 与 Bootstrap Solver 的形式出现。

原则四：架构要做“稳定骨架”，而不要做“能力枷锁”。这一条最容易被忽略，却最具远见。一个 Agent 框架如果包揽太多逻辑——内置任务图、内置策略、内置 Prompt 模板——短期内看似“开箱即用”，但当 LLM 进化出新的能力时，这些写死的逻辑会反过来压制模型的能力。讲者希望团队的架构只提供必要的状态、工具、隔离与反馈闭环，让模型自身的判断力沿着可组合的接口自由生长——所谓“稳定的骨架，而不是能力的枷锁”。

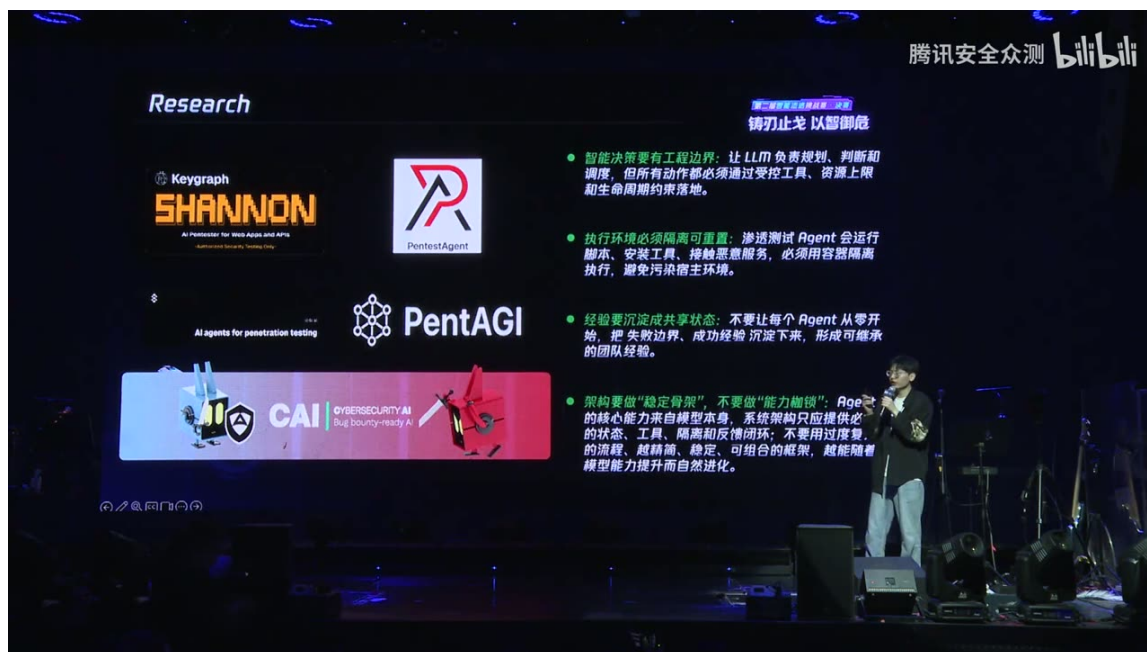


图 1: 幻灯片“Research”：左侧是 Keygraph SHANNON / PentestAgent / PentAGI / CAI 四个开源 Pentest Agent 的 Logo，右侧是团队总结的四条设计理念。这张图是整篇分享的“宪法图”，后续每一节都可对照回查。

把上面四条原则压缩成一段可以随时翻回查阅的“宪法”：

Adaptive Architecture 的四条设计理念

1. 智能决策要有工程边界——LLM 负责规划/判断/调度，所有动作必须通过受控工具、资源上限、生命周期约束落地。
2. 执行环境必须隔离可重置——容器化隔离避免目标反制与本机污染；任意 Solver 出错都可以被重置或丢弃。
3. 经验要沉淀成共享状态——失败边界、有效线索、候选思路写入跨 Agent 的共享存储，新 Agent 启动即继承。
4. 架构要做“稳定骨架”而非“能力枷锁”——只提供状态、工具、隔离与反馈闭环，让模型沿着接口自由生长。

1.4 本章小结

这一节的目的不是介绍系统，而是先确立评判系统的标尺。后文将出现的每一个组件，都可以反过来对照这四条原则去问：它在落地哪一条？

为了让读者对全文结构提前形成索引，我们做一个简短的预告式映射：原则一（智能决策要有工程边界）将在 Manager Agent 的 Policy Constraints 与八件套 Manager Tools 中落地；原则二（执行环境隔离可重置）由 Kali 容器执行平面承担；原则三（经验沉淀为共享状态）由 Solver Memory / Challenge Memory 双层 Memory 与 Observer Sidecar 联手实现；原则四（稳定骨架不做枷锁）则贯穿全文，并在 Harness 四大创新（智能纠偏、Memory & Idea、Ralph-Loop、上下文压缩）中体现得最为明显——Harness 把模型不可控的部分包住，留给 LLM 的只剩“高价值的判断力”。

带着这张“宪法”，我们进入第 2 节，看看团队是怎么把它翻译成一套可在 15 分 55 秒的比赛中、且赛后可复盘的工程系统的。

2 整体架构与工程实践

第 1 节确立了四条设计理念。本节要回答一个工程化问题：怎么把抽象的“宪法”落进一份 Python 代码仓里，并且让这份代码仓既能撑住 15 分 55 秒的比赛极限场景，又能在赛后沿着事后视角逐步复盘？讲者给出的答案分为三层：选一个足够轻、足够优雅的底层 Agent SDK；在它之上做控制面与执行面解耦；最后用一套页面化的工程平台把整套系统包成“可配置 / 可组合 / 可追踪 / 可复盘”的产品形态。

2.1 基础框架：PydanticAI (PydanticAI SDK)

整套系统并不是从 LangChain 之类的成品框架硬改出来的，而是在 **PydanticAI (PydanticAI SDK)** 之上从零搭建。讲者选择 PydanticAI 的原因有两条：一是“整体的实现非常的优雅”——Pydantic 一以贯之的类型化结构让 Agent 状态、消息、工具调用都能用 schema 精确刻画；二是“扩展性也非常高”，便于团队在非常细的颗粒度上插桩、打补丁、定制 Harness。

需要提醒读者的是，讲者在 Q&A 第二轮里把 PydanticAI 描述为“open Cloud (Claude Code) 底层框架”——这是一处口误。

术语澄清：PydanticAI 与 Claude Code / Claude Agent SDK 的关系

讲者口中的“open Cloud / cloud cocodex”应作“Claude Code / Codex”；他将 PydanticAI 称为 Claude Code 的“底层框架”属于现场口误。**技术事实：**Claude Code 实际构建在 **Claude Agent SDK** 之上，与 PydanticAI 是平行的两个生态——前者由 Anthropic 主导，后者由 Pydantic 团队维护。两者在设计思路确有近似之处（都偏类型化、都强调可组合的 Tools 与 Skill），但并不存在归属关系。读者在阅读后续章节时，凡是出现“open Cloud 底层框架”的字眼，请按 PydanticAI 解读即可。

口误之外，“选 PydanticAI”这个决定本身却是非常关键的工程信号：它意味着团队把 Agent 框架视作薄薄的脚手架，而不是“厚重的中台”——这正是第 1 节原则四（架构是骨架不是枷锁）的直接体现。

2.2 控制面 / 执行面解耦

在 PydanticAI 之上，团队做的第一件结构性的事，是把整个系统切成两个平面：左侧是“平台控制平面”，右侧是“隔离执行平面（Kali 容器）”；中间通过“消息总线”串起来；最右还接出一组“外部依赖”。这张图是后文一切讨论的“地图”，请先参照图 2 把整体形状刻在脑子里。

左侧 · 平台控制平面 (Control Plane)。这一侧承担“大脑”职责，包括：

- **统一配置中心**——比赛策略、模型配置、Prompt 模板、可用工具集合都从这里下发。
- **Planner / Manager**——全局任务规划与调度大脑（即第 3 节将展开的 Manager Agent）。
- **Challenge 协调层**——按赛区/赛题维度组织上下文与共享状态。
- **Runtime 调度层**——把策略翻译成对执行平面的具体调度指令（启停 Solver、切换赛题等）。
- **共享协作状态**——跨 Solver 共享的 Challenge Memory / Challenge Idea / Hint / Flag Progress（详见第 4 节）。

中间·消息总线（Message Bus / Unified Channel）。控制面与执行面不是直接互相调用的；中间通过一条统一通道传递“调度指令”与“事件回传”。底层落到通信协议层是 RPC / JSON-RPC，再加一层桥接协议，把控制平面与多个 Kali 容器解耦。这条总线既是控制信号的下行通道，也是 Solver / Observer 状态的上行通道——后者尤为重要，是反馈闭环得以建立的物理基础。

右侧·隔离执行平面（Execution Plane）。每个 Solver 都跑在自己的 Kali 容器里。容器内部成对部署：一个 Solver Session（实际负责攻击执行的 Agent，俗称“打手”）和一个 Observer Sidecar（旁观者侧车，负责状态维护、记忆整理与纠偏判断）。同一时间往往有 A/B/C 三个容器并行——这与第 3 节将提到的“三个并发卡槽”硬约束直接对应。

最右·外部依赖（External Dependencies）。整套系统并不孤立运行，它至少需要三类外部接口：Challenge API（赛题接口，比赛平台提供）、LLM Provider（一个或多个模型服务）、以及若干 MCP（Model Context Protocol）Servers，用以扩展 Agent 可调用的工具集。

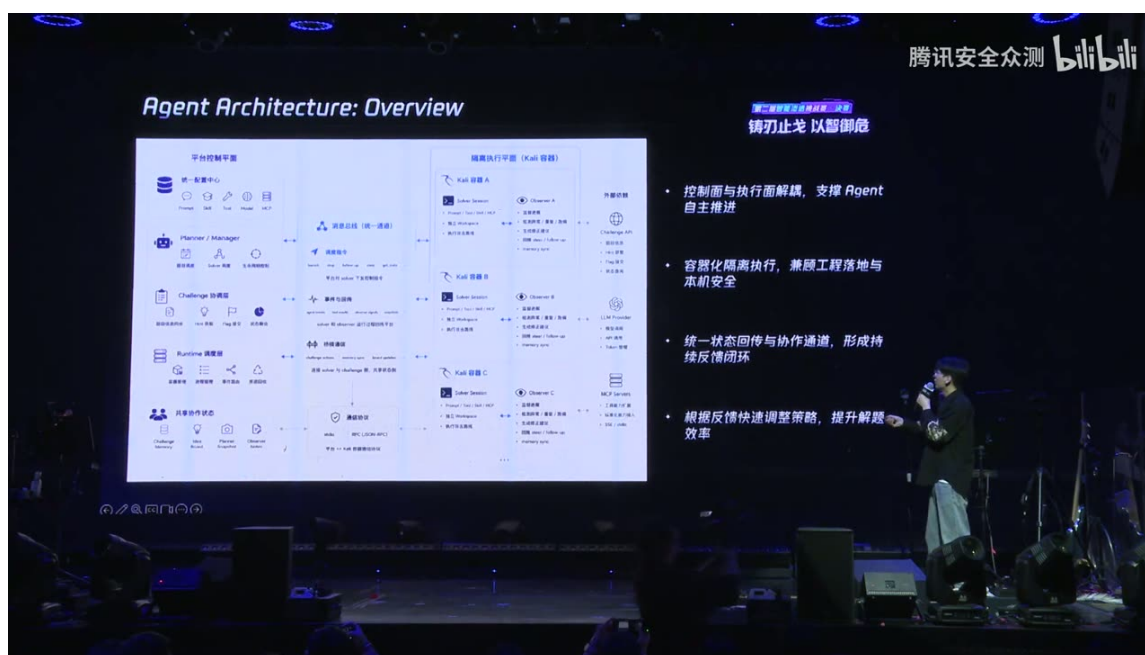


图 2: Adaptive Architecture 总览图：左控制面（统一配置中心 / Planner & Manager / Challenge 协调层 / Runtime 调度层 / 共享协作状态），中消息总线（统一通道 + RPC/JSON-RPC + 桥接协议），右隔离执行面（Kali 容器 A/B/C，每个含 Solver Session 与 Observer Sidecar），最右外部依赖（Challenge API / LLM Provider / MCP Servers）。

为什么要做控制面/执行面解耦？

这是第 1 节原则四（“稳定骨架而非能力枷锁”）的最直接落地：控制面只关心何时调度、调度什么，执行面只关心在隔离沙箱里把动作跑出来。两侧通过消息总线对话，任何一侧都可以独立替换或扩展——可以加新的 Solver 容器、可以换 Manager 的策略实现、可以接入新的 LLM Provider 或 MCP Server，而不会牵动另一侧。骨架越简洁，模型越能在它之上自由生长。

2.3 Kali 容器隔离：为什么必须容器化

控制面/执行面解耦只是结构上的拆分；执行面之所以选择 **Kali Linux** 容器作为载体，背后还有更具体的安全与工程考虑。

为什么不让 Solver 跑在宿主机上？

- **目标反制风险**——渗透测试 Agent 主动与可疑目标通信、执行 payload；一旦目标具备反制能力（恶意响应、回连利用），缺乏隔离的宿主机就是直接靶面。
- **本机环境污染**——攻击工具会装包、改配置、留临时文件、起守护进程；多次任务叠加之后，宿主环境会被悄悄改变，污染下一次实验，也威胁本机安全（Host Security）。

容器化隔离让每一个 Solver 都有自己的、可以随时丢弃并重置的执行环境，恰好对应第 1 节原则二“执行环境必须隔离可重置”。

更具体地说，A/B/C 三个 Kali 容器在比赛中同时存在，每个容器内部都是“Solver Session + Observer Sidecar”的成对结构。当 Manager 决定停掉某条线、切换赛题路线时，相应容器可以被重置而不会影响其他容器；这也使得整套系统在面对失败时能够快速止损——这是后续 Manager 调度策略得以激进的物理前提。

2.4 消息总线与统一状态回传

控制面/执行面解耦带来一个直接的工程问题：控制面如何知道执行面发生了什么？答案是**消息总线上的统一状态回传**。

具体来说，每个容器中的 Solver 与 Observer Sidecar 都会通过消息总线把事件实时上报给控制面：Solver 上报当前推进步骤、工具调用结果、是否拿到 Hint、是否达到 Flag；Observer 上报对当前 Solver 是否漂移（Drift Detected）、新发现的有效线索、值得提出的假设。控制面收到这些事件之后，会驱动 Manager 做出一组调度决策——讲者用“何时去停掉我们的 Solver”与“何时去调整题目的动态轮换”两个具体动作来描述这层闭环。

也就是说，消息总线不是一条被动的数据通道，而是**反馈闭环的物理载体**。它把执行面的实时状态变成控制面的输入；控制面再把策略调整变回执行面的指令——一旦闭环跑起来，整体系统就具备了“根据现场动态调整策略”的能力，而不是按一份静态任务图机械执行。这套闭环具体如何驱动 Manager 的调度决策，将在第 3 节系统展开。

2.5 工程实践四要素：从架构到产品

到这里，整套架构在概念上已经搭好。但讲者特别强调的一点是：**Adaptive Architecture** 不只是一张架构图，更是一套可以在浏览器里点击操作的工程平台——这是“工程化”与“学术原型”的分水岭。这套平台围绕四个工程目标展开：

- **策略可配置（Configurable Policy）**：比赛中所有的策略——调度（Planner 超时时间、调度偏好）、模型选择、Prompt 模板、工具集合——都可以在页面上动态调整，无需改代码、无需重启容器。
- **Agent 自由组合（Composable Agent）**：一个 Agent = LLM + Prompt + Tools + Skills + Observer。每一个原子组件都能在页面上灵活调配，主 Agent 与 Observer Agent 的配对也是

可组合关系。

- **过程可追踪 (Traceable Process)**: Solver 日志、Memory、Idea、提交记录全量留痕——比赛中的每一个动作、每一次状态更新都被记录下来，且可以按 Challenge / Solver 维度回溯。
- **结果可复盘 (Replayable Outcome)**: 除了把过程数据存下来，平台还提供专门的“复盘页面”，可以从拿 Flag 的瞬间倒着回溯整条路径——花了多少 token、走了哪条路、关键决策出现在哪一步。

为什么这四点不是“锦上添花”而是“地板要求”？因为渗透测试本质上是一个无人值守 (Unattended Operation) 场景：比赛中的 15 分 55 秒里，选手不能干预 Agent 的具体决策；只有在赛前充分配置、赛中持续监控、赛后彻底复盘，团队才能把每一次提交都转化为下一次迭代的输入。

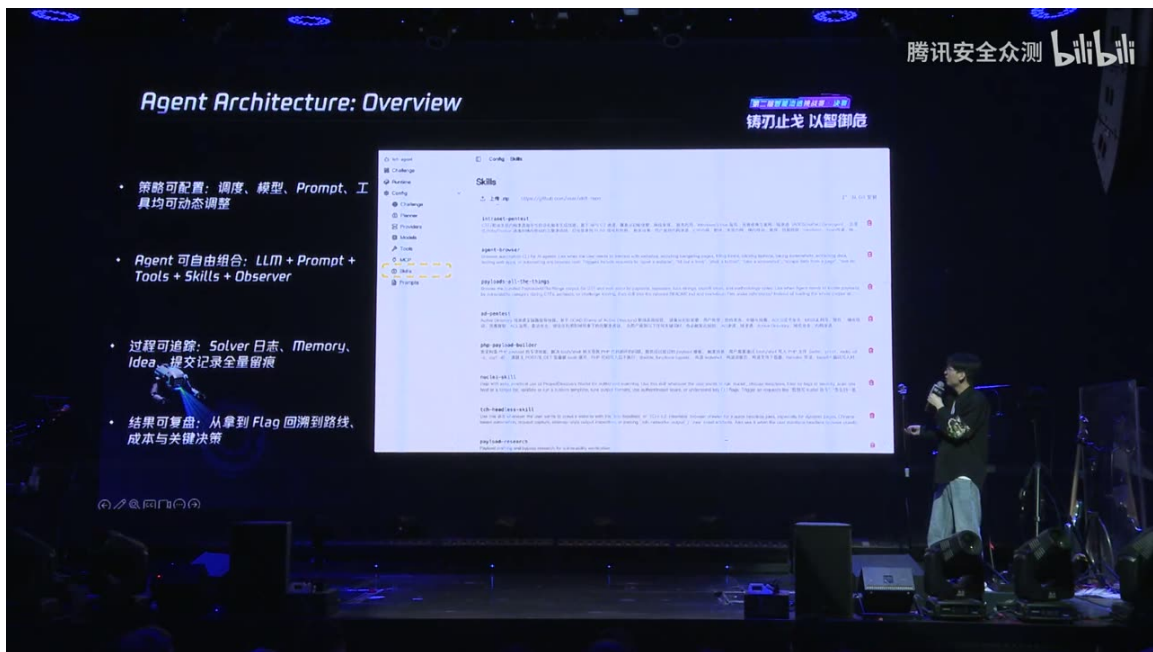


图 3: 工程平台的 Skills 配置页面：左侧菜单覆盖 Challenge / Runtime / Config (含 Challenge / Planner / Providers / Models / Tools / MCP / Skills / Prompts)，主区域是 Skills 列表 (init-pentest / agent-browse / payloads-all-the-things / ad-pentest / php-payload-builder / nuclei-skill / payload-research 等)。这一页直观地说明了“Agent 自由组合”的载体——Skills 是可挂载、可替换、可组合的能力单元。

2.6 本章小结

回到第 1 节那张“宪法”，本节其实是它的逐条工程化映射：

- **原则二 (执行环境必须隔离可重置)**——由 Kali 容器执行平面承担：A/B/C 多个容器并行，可重置、可丢弃，避免目标反制与本机污染。
- **原则四 (架构要做稳定骨架而非能力枷锁)**——由 PydanticAI + 控制面/执行面解耦 + 消息总线三件事联手承担：底层 SDK 薄、上层结构正交、组件间通过消息总线松耦合，留出最大的演化空间。
- **原则一 (智能决策要有工程边界)**——本节只画出了边界的“地基” (Runtime 调度层、消息总线、可配置策略)；真正的智能决策机制将在第 3 节由 Manager Agent 完整呈现。

- **原则三（经验要沉淀成共享状态）**——本节只露出了“共享协作状态”这一格局；它的具体内部结构（Challenge Memory / Challenge Idea / Hint / Flag Progress 与 Solver 局部 Memory 的两层关系）将在第 4 节展开。

可以这样总结：第 2 节给出的是一副“可以装东西的骨架”；第 3 节起，我们开始往骨架里装真正的智能。

3 Manager Agent：调度与边际效益最大化

整体架构与工程实践讲完之后，演讲者把焦点切换到整套系统的”大脑”——Manager Agent。这一节回答的是一个非常具体、却又非常关键的工程问题：在比赛硬约束下，怎样让每一份算力、每一颗 Token、每一秒墙上时间都花在最值得的赛题上？我们将沿着”约束 → 上下文 → 决策原则 → 工具集 → 输出动作”的顺序，逐层拆解 Manager Agent 的设计骨架。

3.1 比赛硬约束：三个并发卡槽

第二届腾讯云黑客松的智能渗透挑战赛，给参赛队伍设了一道刚性约束：每个赛区同时 ** 只允许三个并发的 Solver 卡槽 **（Three-Slot Constraint）在线工作。这条规则看似平淡，却把整场比赛瞬间转化成一道经典的”有限资源下的最大化问题”——你只能同时点亮三盏灯，每一盏灯往哪个赛题上照、什么时候熄灭、什么时候换题，都会直接决定 Flag 总量与排名。

换句话说，Manager Agent 的核心目标不是”把所有题都做一遍”，而是让 ** 每一个卡槽的边际效益最大化 **。一旦某个 Solver 在赛题 X 上长时间没有产出新的有效信号，继续占用卡槽就是在向其它赛题”借时间”——必须果断停掉、置换更高价值的赛题。

核心命题：三卡槽是硬约束，调度即生产力。Manager Agent 的所有设计选择，都必须围绕同一个目标展开——在三个并发卡槽内，让单位 Token 与单位时间所产出的 Flag 数量最大化。

这条约束把”调度”从工程小事拔高为整场比赛的胜负手，也直接催生了下文 LLM 与工程之间的责任划分。

3.2 从战场感知到动态规划

为什么传统的”轮询调度（Round Robin）”或”先到先服务（FIFO）”在这里完全不适用？因为渗透赛题之间存在巨大的”价值波动”：同一个 Solver，在赛题 A 上半小时一无所获，切到赛题 B 也许十分钟就能拿到一血；某条线索可能让原本死局的赛题瞬间盘活。静态排队会无差别地浪费卡槽，而 Manager Agent 必须像一位”战场指挥官”那样，从全局上下文中实时感知战况、动态做出规划。

为此，演讲者把”战场感知”形式化成了 八路调度上下文（Scheduling Context）——这八路输入构成 Manager Agent 每一次决策的事实底盘：

- **赛题总体状态**：哪些赛区已开放、哪些尚未解锁、各赛题的悬赏与剩余次数。
- **Solver 运行进度**：每个在线 Solver 已执行多少轮、最近在做什么、是否在原地打转。
- **Observer 反馈**：旁观者侧车（Observer Sidecar）对当前推进有效性的判断。
- **Challenge Memory**：全局已固化的事实与边界。
- **Idea Board**：当前值得继续推进的攻击假设。
- **历史尝试结果**：以往哪条路径已被证伪、哪条仍待验证。
- **工具执行结果**：来自 Kali 容器内工具调用的最新返回。
- **当前资源占用**：Token、墙上时间、容器算力的瞬时消耗。

把这八路输入合并起来，Manager Agent 等价于在做一次”小型 OODA 循环”：观察（Observe）→ 判断（Orient）→ 决策（Decide）→ 行动（Act），并且每一轮的”行动”都会立即反馈到下一轮的”观察”，形成自适应闭环。

3.3 LLM 做规划，工程做边界

如果说三卡槽约束定义了”问题”，调度上下文定义了”输入”，那么”LLM 做规划，工程做边界”就是 Manager Agent 的 **第一性原则**。这八个字浓缩了演讲者反复强调的设计哲学：把灵活性留给智能体，把可控性留给工程系统。

原则：LLM 做规划，工程做边界。

- **LLM 负责灵活性**：基于全局上下文，做赛题选型、路线选择、优先级排序、阶段判断。这部分需要”判断力”和”经验直觉”——交给模型最合适。
- **工程负责可控性**：通过受限的 Manager Tools 把策略转化为可执行的调度动作；通过 Harness/Policy Constraints 限定最大并发数、可用工具集、生命周期与资源预算。这部分需要”确定性”和”边界感”——交给系统最合适。

这条原则同时是”智能决策要有工程边界”四大设计理念在 Manager Agent 上的最直接落点。

落到工程上，这条原则有一个非常具体的安全意义：即便 LLM 因为幻觉、上下文腐败或恶意 Prompt 注入而产生异常决策，工程层的工具白名单、资源上限和生命周期约束依然能把损失锁死在可控范围之内——卡槽不会突然冒出第四个，Solver 不会无限烧 Token，宿主机不会被污染。

3.4 Manager Tools 八件套

如何把”LLM 的判断”翻译成”工程可执行的动作”？答案是把 Manager Agent 的写权限收敛成一组 **八件套工具**（Manager Tools）。每个工具都是一条受控、可审计、可回放的调度指令——这正是 fig-manager-design 幻灯片右半侧所展示的接口面板。

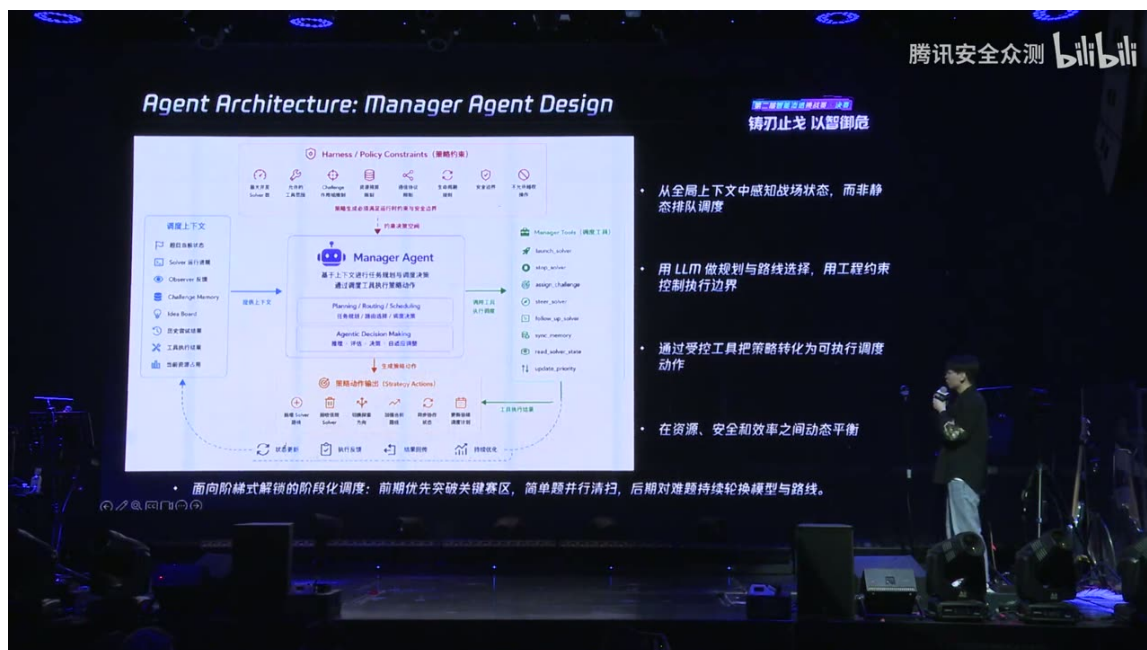


图 4: Manager Agent Design: 顶部 Harness/Policy Constraints 给出工程边界; 左侧八路调度上下文构成战场感知; 中央 Manager Agent 做 Planning/ Routing/ Scheduling 与 Agentic Decision Making; 右侧 Manager Tools 八件套把判断转换为受控动作; 底部 Strategy Actions 六件套是最终的策略输出。

Manager Tools 八件套（每条解释其触发时机）:

- **launch_solver**: 决定开新卡槽时，启动一个新的 Solver Session 并附带 Bootstrap 状态。
- **stop_solver**: 判断某 Solver 已无边际价值时，回收其卡槽以让位更高价值赛题。
- **assign_challenge**: 在 Solver 与赛题之间建立“路由”，决定其攻击目标。
- **steer_solver**: 在 Solver 偏离主航向时，注入轻量的纠偏指令而不打断主流程。
- **follow_up_solver**: 基于最新进度向 Solver 追加上下文或下一步指令。
- **sync_memory**: 把 Observer 整理后的局部状态合入全局共享态，触发跨 Solver 同步。
- **read_solver_state**: 在不打扰 Solver 主流程的前提下读取其当前运行快照。
- **update_priority**: 根据时段、剩余赛题与已得 Flag 数，动态调整赛题与 Solver 的调度优先级。

这八件套合起来，是 LLM 唯一能写入系统的“接口面板”——所有动作都受 Harness/Policy Constraints 的进一步限制（最大并发、工具白名单、生命周期、资源预算等）。

3.5 Strategy Actions 六件套：六类策略输出

Manager Tools 是”原子动作”，那么 Manager Agent 在每一次决策周期里实际产生的”策略层输出”则被归纳为 六类 **Strategy Actions**，与幻灯片底部的输出带一一对应：

- **新建 Solver 路线**：开辟新攻击线，对应 `launch_solver + assign_challenge` 的组合。
- **剧停 Solver**：果断回收低值卡槽，对应 `stop_solver`。
- **切换赛题路线**：保留 Solver 但重新指派目标，对应 `assign_challenge` 切换。
- **加微执行轨迹**：通过 `steer_solver / follow_up_solver` 微调当前推进。
- **同步状态**：把 Observer 沉淀的局部经验经 `sync_memory` 合入全局共享态。
- **更新调度优先级**：调用 `update_priority` 重排赛题/Solver 权重，呼应阶段化解锁的时序策略。

这六类输出与”阶段化解锁”的赛事节奏天然契合：前期优先突破关键赛区以解锁后续题面；中期把简单题并行清扫，最大化 Flag 量产速度；后期对剩余难题持续轮换模型与路线，以微调代替整体重启。

3.6 本章小结

把这一节的逻辑链条收拢起来：硬约束（三卡槽）→ 输入（八路调度上下文）→ 原则（LLM 做规划，工程做边界）→ 接口（Manager Tools 八件套）→ 输出（Strategy Actions 六件套）。Manager Agent 因此成为整套 Adaptive Architecture 中”工程边界 + 智能决策”原则最具代表性的落点：它既不让 LLM 直接接触底层资源，也不把决策硬编码成确定性脚本，而是用一组受控工具，把 LLM 的判断力翻译成可审计、可回放、可中断的调度动作，最终在”资源安全”与”执行效率”之间动态求解出当下最优的卡槽分配。

下一节我们将进入卡槽内部——Solver Agent 究竟如何在并行探索中，把每一次失败也沉淀成下一次突破的起点。

4 Solver Agent：并行探索与共享状态

Manager Agent 决定了”派谁去打哪道题”，而真正下场厮杀的是 Solver——演讲者也亲切地称之为”打手”。三个并发卡槽意味着同一时刻最多三个 Solver 在攻击不同赛题，它们应当如何独立工作而不互相干扰？又如何让每一次失败的尝试不被白白丢弃？这一节回答的核心问题是：**多 Solver 并行探索时，怎样既隔离又协同？**

4.1 第一性原则：探索不应消失

整个 Solver 子系统的设计，被演讲者用一句话浓缩成第一性原则——

每一次探索都不应消失，而应沉淀为下一次突破的起点。

这不是一句口号，而是一条具体的工程约束：任何 Solver 的失败路径、被证伪的假设、扫到的中间产物，都不允许”随着 Solver 退出而蒸发”。系统必须有一条机制把局部经验过滤、提纯、固化，最终回流到全局，供后来者继承。这就是 Solver 设计与 Observer Sidecar 协作的全部出发点。

之所以要把这条原则提到第一位，是因为渗透测试天然是高失败率任务：扫描大多无果、Payload 大多不打、注入大多被过滤；如果每个 Solver 死掉之后什么都不留，下一个 Solver 又得从零摸索同一片雷区，那么 Token 与时间就会被反复浪费。把”探索”沉淀成”经验”，本质是让全队的有效信息熵随着时间单调上升。

4.2 并行探索的隔离设计

实现这条原则的第一步，是把 Solver 之间隔离开——这听起来与”共享经验”相矛盾，却恰恰是经验得以高质量沉淀的前提。

每一个 Solver 都拥有自己的三件套局部状态：

- **Solver Session**：与 LLM 的会话上下文、当前轮次的工具调用与返回。
- **Solver Memory**：该 Solver 在当前赛题上观察到的事实、证据与尝试边界。
- **Solver Idea**：该 Solver 当前持有的攻击假设与候选思路。

关键的工程细节是：这三件套被明确标注为 **Local only / Not broadcast**——它们不会被直接广播给其它 Solver，更不会立刻进入全局共享态。理由很现实：渗透是一个充满”伪信号”的过程——一个 Solver 在赛题 A 上的”看似有效线索”放到赛题 B 上可能完全是噪音；某个 Solver 还在测试期的假设如果立刻广播，会污染所有兄弟 Solver 的判断。隔离是为了让每一个 Solver 在足够小的”局部一致性领域”内独立推进，避免互相串味。

4.3 Observer Sidecar：旁观者沉淀

可是局部状态如果只是”局部”，第一性原则就无从谈起。于是每个 Solver 都被配上一只 **Observer Sidecar**——以服务网格中的”侧车”模式（Sidecar Pattern）作类比：旁观者与 Solver 同生共死，但不参与攻击，只负责”看与提炼”。

Observer Sidecar 持续观察其所附 Solver 的会话流、工具反馈与状态变化，从中提炼出三类高价值信号：

- **失败边界**：哪些路径已被证伪、哪些 Payload 已被过滤——避免兄弟 Solver 重复踩坑。
- **有效线索**：哪些工具响应里出现了潜在可利用信号，例如版本号、目录、错误码模式。
- **候选思路**：基于当前局部状态归纳出的、值得后续验证的假设。

这一层”旁观者沉淀”，等价于为每个 Solver 配置了一名”随队记者 + 战术分析师”：Solver 专注作战，Observer 专注复盘，二者互不打扰却共同推进经验固化。这也是为什么 Observer 的存在被演讲者称为”经验沉淀原则的执行单元”。

4.4 两层 Memory：局部 vs 全局

把 Solver 局部状态与 Observer 沉淀结果合起来看，整个 Solver 子系统其实建立了一个**两级记忆层级（Memory Hierarchy）**：

两层 Memory 设计：

- **局部层（Per-Solver, Local only）**：Solver Session + Solver Memory + Solver Idea，三件套绑定单个 Solver，不直接广播。
- **全局层（Shared State）**：四件套构成全队共享的”作战看板”——
 - **Challenge Memory**：全局已确认的发现与共享事实。
 - **Challenge Idea**：跨 Solver 的共享假设与候选思路。
 - **Hint**：从赛题或工具反馈中提取的线索情报。
 - **Flag Progress**：当前各赛题的拿 Flag 进度。

局部层提供”高带宽、低延迟、低噪音”的个体推理环境；全局层提供”经验跨界继承”的团队基础设施。两层之间的桥梁，就是 Observer Sidecar——只有经过 Observer 提炼的局部经验，才有资格被合入全局，再由全局广播回所有 Solver。

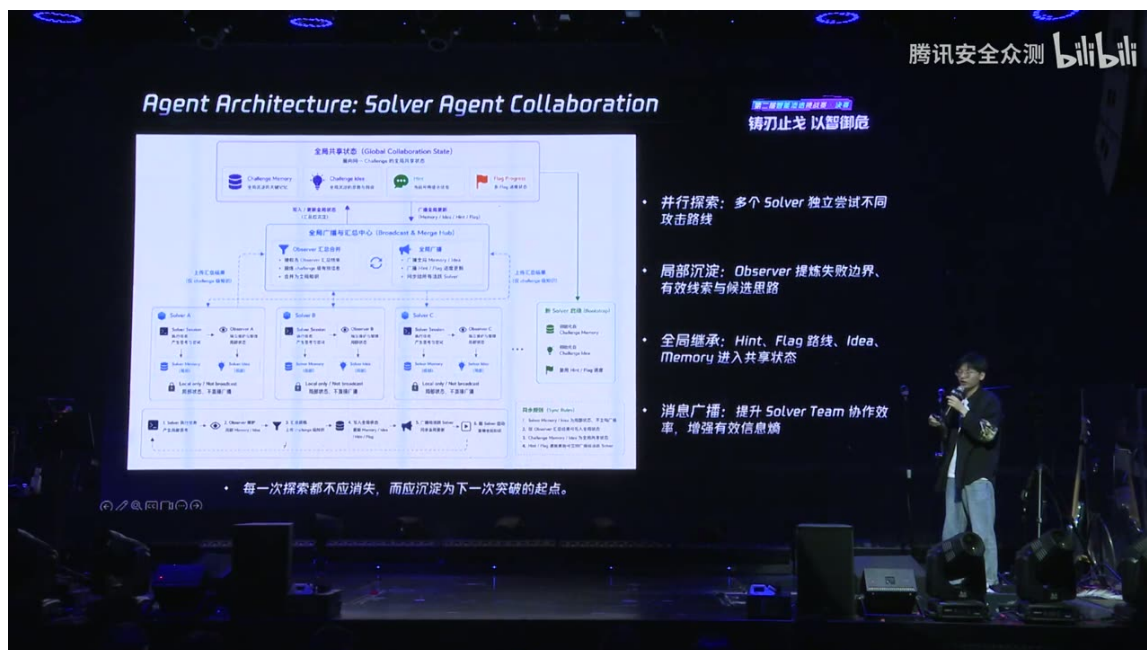


图 5: Solver Agent Collaboration: 顶部为全局共享态 (Challenge Memory / Challenge Idea / Hint / Flag Progress), 中部 Observer 汇总并合并各 Solver 的局部状态后向全局广播; 左下三个 Solver A/B/C 并行运行, 各自携带 Solver Session + Solver Memory + Solver Idea 并标注为 Local only / Not broadcast; 右下”新 Solver 启动 (Bootstrap)”从全局共享态拉取初始知识; 底部 Sync Rules 同步规则带统一调度。

4.5 广播机制: 信号 → 消息总线 → 所有 Solver

有了两层 Memory, 剩下的问题是: 经验如何在全队之间流动? 答案是基于”信号驱动”的广播机制。

当某个 Observer Sidecar 在所附 Solver 上识别到**足够确定**的有用信号——例如某条 SQL 注入路径被验证有效、某个未公开端口被发现、某条线索能够跨赛题复用——它会:

1. 把局部信号汇总为”可对外发布”的事实/假设/线索;
2. 通过消息总线 (Message Bus) 向上提交, 与全局共享态合并;
3. 由全局广播回所有在线 Solver, 让兄弟 Solver 在下一轮 Session 中即刻受益。

这条机制还顺带解决了”新 Solver 怎样起步”的问题: 当 Manager 调用 `launch_solver` 启动一个 **Bootstrap Solver** 时, 新 Solver 会先从全局共享态拷贝当下的 Challenge Memory、Challenge Idea、Hint 与 Flag Progress——它一上线, 便已经”踩在前辈的肩膀上”, 而不必从零摸索。

为什么记忆不能直接共享，必须经 Observer 过滤再广播？

- **噪音过滤**：Solver 的原始 Session 中混杂大量伪信号、试探失败、工具异常输出；直接共享会让所有 Solver 一起淹没在噪音里。
- **假设隔离**：处于“Pending / Testing”阶段的 Idea 还未被证实，过早广播会让其它 Solver 基于错误假设做无用功。
- **语义对齐**：不同 Solver 的局部命名、上下文不尽相同，Observer 负责把局部表达转写成全局统一术语。
- **合并去重**：同一线索可能被多个 Solver 各自发现，Observer 在合并入全局前完成去重与冲突仲裁。
- **安全闸门**：Solver 对全局共享态只读不写——写权限被收敛到 Observer 与 Manager，避免任何单个 Solver 把整个团队带偏。

简而言之，“经过 Observer 再广播”既是降噪机制，也是记忆质量的护城河；缺了这一层过滤，“经验沉淀”反而会变成“错误污染”。

4.6 本章小结

整个 Solver 子系统沿着同一条主线展开：先用“局部隔离”把每一个 Solver 的推理环境保持干净，再用 Observer Sidecar 把每一次失败与中间产物提炼成可复用的事实/假设/线索，最后用“信号 → 消息总线 → 全局共享 → 广播 / Bootstrap”的链路，把局部经验回流给全队。Observer Sidecar 在其中扮演了“经验沉淀原则”的执行单元：它既保证 Solver 的局部一致性，又保证全局共享态的高信噪比。

回到本节开篇那句第一性原则：

每一次探索都不应消失，而应沉淀为下一次突破的起点。

至此，从 Manager Agent 的卡槽调度，到 Solver Agent 的并行探索与经验沉淀，整套 Adaptive Architecture 的“控制层”已经讲完——下一节将由队友接力，深入到 Solver 内部的稳定性骨架：智能纠偏、双层状态板、Ralph-Loop 终止控制环，以及上下文 Harness 的压缩链路。

5 Harness 四大创新：Solver Harness 与 Context Harness

从字幕第 139 句开始，演讲进入由队友 B 主讲的下半场。如果说前面四节给出的是“由谁来做、在哪里做、做什么”——Manager 调度、Solver 并行、Observer 沉淀——那么本节回答的是一个更工程的问题：在长任务、噪音多、模型上下文易腐败的渗透场景下，如何让 Solver “稳得住、不偏移、不早停”？

答案是 Harness。讲者在这一段用四个并列的“创新点”把 Harness 的内涵展开：智能纠偏（Efficiency Reminder）、Memory & Idea 双层状态板、Ralph-Loop 终止控制、上下文压缩（Context Harness: Compaction）。前三者属于 **Observer Harness** ——围绕 Solver 的“侧车”侧面进行干预；最后一项属于 **Context Harness** ——直接对模型每一次调用所“读”到的内容做加工。这四点合在一起，就是这套架构区别于 Claude Code 范式的核心增量。

5.1 Harness 框架：稳定运行的四个支柱

回到第 1 节给出的“四条设计理念”——智能决策要有工程边界、执行环境必须隔离可重置、经验要沉淀成共享状态、架构要做“稳定骨架”而不是“能力枷锁”。前三条已经在 §2 的容器化平面、§3 的 Manager 调度、§4 的 Solver/Observer 协作中各自落地；剩下“稳定骨架”这条原则，恰恰要靠 Harness 来兜底。

讲者在第 142 句把这一段的整体定位说得很清楚：“我来讲一下我们 Solver Agent 的 Harness 工程实践和创新点。”Harness 不是新增功能，而是一组围绕 Solver 长程稳定性的工程化护栏。具体落点正是下面四个支柱：

Solver/Context Harness 的四大支柱：

1. **智能纠偏**（Efficiency Reminder / Steer Reminder）——由 Observer Sidecar 持续审查 Solver 推进有效性，在偏离时注入轻量纠偏。
2. **Memory & Idea 双层状态板**——Idea 维护“假设”，Memory 沉淀“事实”，由 Observer 持续整理与压缩。
3. **Ralph-Loop 终止控制**——把“任务是否结束”的判断权从模型主观抽离，改由系统状态约束。
4. **Context Harness: Compaction** ——四步流水线（Raw CLI Output → RTK Rewrite → Context Compression → Solver Context），主上下文只留摘要 + 索引。

合在一起，它们构成“智能渗透 Agent 的稳定运行环境”。

5.2 创新一：智能纠偏（Efficiency Reminder）

5.2.1 痛点：长任务里的“漂移”

讲者描述了一个非常具体的现象：在长任务中，Agent 容易“被近期上下文与工具反馈牵引，执行策略逐步发生偏移”——简而言之就是 **Drift**。这种漂移不是模型能力差，而是注意力机制天然的近端偏置：当你已经看了几十轮 nmap、gobuster、curl 输出之后，模型会被最近的工具结果牵着走，逐渐忘掉真正应该突破的高价值目标。

为什么 Llama 类模型在这一点上更脆弱？演讲第 156 句明确说：“这个价值在于，它天然弥补了 Llama 执行任务的短板。”与 Claude/GPT 系顶级模型相比，Llama 系开源模型在长任务的目标稳定性上明显更弱（容易被工具反馈带偏、提前收尾、漏掉证据），因此对“外部纠偏”的需求更强。Efficiency Reminder 的存在让 Llama 在该架构下也能跑出可比的成绩。

5.2.2 机制：Observer 当裁判，但不当主角

智能纠偏的关键设计在于角色分工：**Observer Agent** 不直接干预主 **Solver**。它只做三件事

- 持续审查近几轮的执行表现与工具调用反馈（典型窗口为最近的 R5 / R6 / R7 / R8 几轮）；
- 基于目标判断“当前推进是否仍然有效”（Drift Detected 状态机告警）；
- 在适当时机以 **Steer Reminder**（轻量提醒）打断或纠偏，把判断结果写回主 Solver 的上下文，但不替主 Solver 决策。

这是典型的 **Observer-as-Judge** 模式：把“判断推进是否有效”这种昂贵的元认知任务从主 Solver 卸载到旁观者，主 Solver 仍然只负责“做事”，但被旁观者拉一把就不会跑偏。

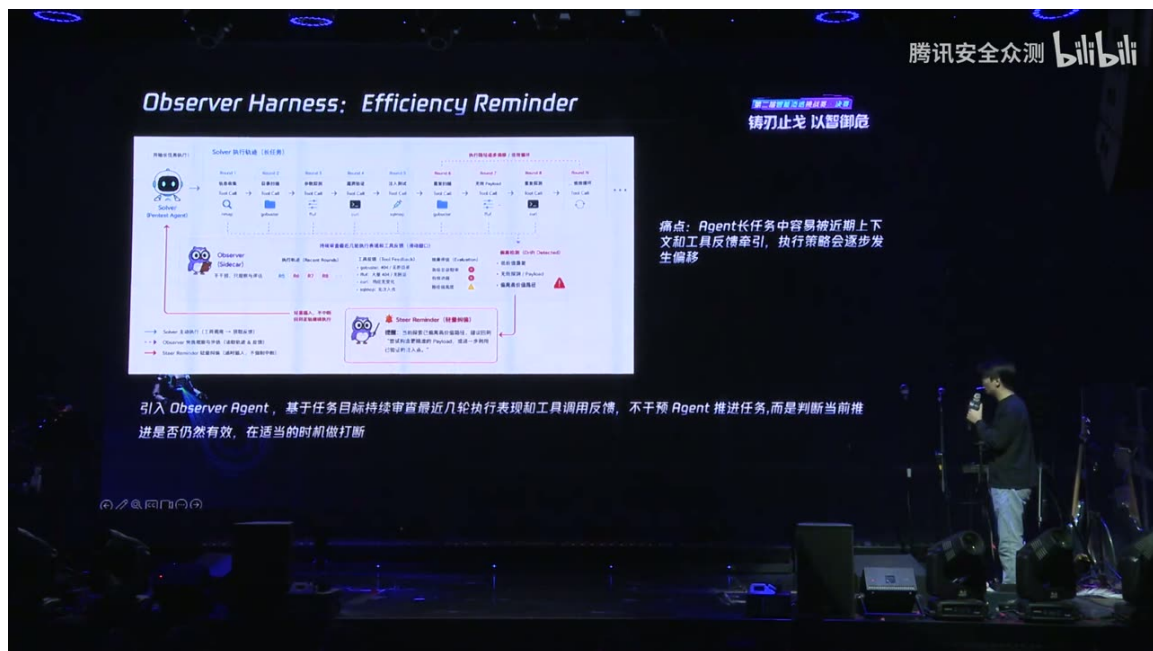


图 6: Observer Harness: Efficiency Reminder。上方为 Solver 长任务执行轨迹 (R1..N, 含 nmap / gobuster / curl / sqlmap 等工具调用)，中部 Observer Sidecar 持续审查最近几轮反馈 (R5-R8)，右下“Steer Reminder”卡片以红色虚线纠偏拉回主轨迹。

1

¹ 视频画面时间区间：00:06:38 (frame_0200)。

5.3 创新二：Memory & Idea 双层状态板

5.3.1 真正失去的不是能力，而是状态本身

第 161–164 句是这一段的 motto：“长任务推进到后期，真正失去的不是能力，而是状态本身——假设、证据和失败边界过程日志中混在一起。”这句话点出了渗透 Agent 的真正难点：模型的能力在每一轮调用都是相同的，但当上下文里堆满了几十轮的中间结果、错误尝试、半成功的猜测，模型很难再从混杂状态中重建“我现在到底走到哪一步”。

解决思路是——把“假设”和“事实”分开放。Idea Board 装策略层的方向与假设，Memory Board 装事实层的证据与边界。这两块都不再由 Solver 自己维护，而是交给 Observer Sidecar 在每一轮之后整理、合并、压缩。

5.3.2 Idea Board：策略层的状态机

Idea Board 维护“当前值得推进的攻击方向与假设”，每条 Idea 都是状态机里的一个节点：

- **Pending**：候选假设刚提出，尚未验证；
- **Testing**：主 Solver 正在执行验证中；
- **Verified**：假设被证实，沉淀进入 Memory Board；
- **Failed**：假设被证伪，写入 Failure Boundary，避免后续 Solver 走回头路。

注意 Verified 与 Failed 都是有归宿的终止态：Verified 转化成 Memory 里的 Evidence，Failed 转化成 Memory 里的 Failure Boundary。Idea Board 因此是一个“流动的”假设池，长期不会膨胀。

5.3.3 Memory Board：事实层的四类槽位

Memory Board 收纳的是“可跨轮复用的事实”，分四类：

- **Evidence（证据）**：已验证为真的事实，例如目标开放的端口、可访问的 URL、有效的注入点；
- **Failure Boundary（失败边界）**：已被证伪的尝试边界，例如某个端口确认无服务、某个目录确认 404；
- **Hint & Clue（线索情报）**：尚未直接验证、但具有强提示价值的信息（题面提示、前向 Solver 留下的方向）；
- **Constraint（约束条件）**：环境/规则约束，例如不允许的操作、生命周期上限。

核心区分：Memory = 事实，Idea = 假设。两者最大的不同不在内容，而在置信度与可写时机：

- Idea 是“待验证假设”，可以多、可以错，由 Observer 自由提出；
- Memory 必须由 Observer 观察到 Solver 的实际执行结果被证实之后才允许沉淀。

这层“有门槛的写入”就是 §6 评委追问的“Memory 不会无限增长”的第一道闸门。



图 7: Observer Harness: Memory & Idea。左侧”原始状态流”为混乱日志（工具输出 / 过程日志 / 上下文碎片 / 环境约束 / 外部输入），中央双层状态板上层 Idea Board（Pending / Testing / Verified / Failed），下层 Memory Board（Evidence / Failure Boundary / Hint & Clue / Constraint），右侧 Observer Sidecar 持续整理 / 更新 / 压缩 / 合并，最右 Solver 接收”低噪音可决策”上下文。

2

5.4 创新三：Ralph-Loop（终止控制环路）

5.4.1 字幕里的”REFLOOP”是错听

字幕错音提醒：自动字幕在第 173 句把这一节的名字写成了”REFLOOP”，这是 ASR 把”Ralph-Loop”错听了。讲者口误时音节并不清晰，而幻灯片标题清楚写着 **Observer Harness: Ralph-Loop**。本笔记一律以幻灯片为准。

5.4.2 为什么”该不该结束”不能交给模型自己判断

讲者在第 175–180 句给出了 Ralph-Loop 的存在理由：”长任务执行中，Agent 可能因误判、停顿或上下文退化而过早结束任务。”把”任务是否完成”的最终判断交给当下这一轮的模型，等于把整支 Solver 的命运绑在最不稳定的一个变量上：

- 上下文已经被压缩了几次，关键证据可能已被裁掉；
- 当前轮次的工具反馈一旦过于”完美”，模型就会幻觉”任务已完成”；
- 一旦遇到死路，模型的避害倾向会让它倾向于尽早 `end_task`，把成本风险甩给系统。

Ralph-Loop 的核心动作是把”任务是否结束”的决策权从模型主观层抽离出来，改由系统基于任务状态统一约束。当 Agent 试图 `end_task` 时，系统会先做一道任务完成性检查：”目标是否真

²视频画面时间区间：00:06:58（frame_0210）。

的达成？证据是否充分？是否仍有未完成的子目标？”——任何一项不通过，都拒绝结束，把 Solver 拉回主线继续推进。

5.4.3 终止由”外部条件”接管

讲者在第 182–184 句举了三类典型的”外部终止条件”，对应三种工程上可观测、可计算的状态：

- 执行时间到达上限（time-up，资源约束维度）；
- 路径触达（path-reach，目标维度，例如 Flag 文件被读取、关键服务被打通）；
- 任务完成（task-complete，业务维度，例如 Flag 已成功提交）。

幻灯片底部的”绑定作业模板”四块图标对此做了进一步细化：目标维度约束 / 阶段维度约束 / 时序一致性约束 / 终止权限约束——把”何时可以喊停”严格控制在系统侧，而不是模型侧。

Ralph-Loop 的核心原则：

结束条件由任务状态决定，而不是由模型当下的主观判断决定。

这是把 Agent 从”高方差的主观决策机”重新固定为”在工程边界内可靠运行的执行单元”的关键设计。它与第 1 节”智能决策要有工程边界”原则严格对应。

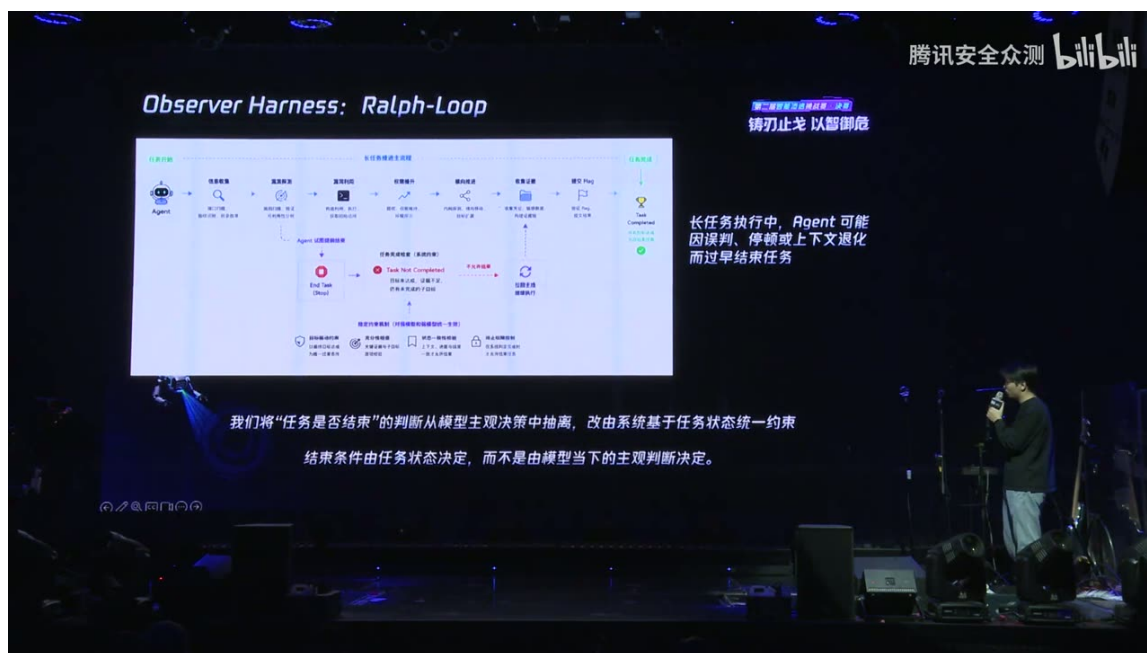


图 8: Observer Harness: Ralph-Loop。主线流水线（任务开始 → 信息收集 → 漏洞探测 → 漏洞利用 → 权限提升 → 横向推进 → 收集证据 → 提交 Flag → 任务完成）；中部分支显示当 Agent 试图 End Task 但系统约束未满足时，被强制拉回主线；底部”绑定作业模板”四块约束图标。

³视频画面时间区间：00:07:48（frame_0235）。

5.5 创新四：Context Harness 之 Compaction 四步流水线

5.5.1 上下文腐败的两大来源

上下文腐败（Context Corruption）警示：第 188–190 句明确指出：“长任务中的上下文压力，主要来自大量的原始工具输出和历史会话堆积，造成上下文腐败导致模型降智。”在渗透场景下尤其严重——单一次 `nmap -sV -p-` 就能输出几千行带颜色控制符、版本字符串、异常 banner 的”AI 不友好”内容；如果不加处理直接灌进主上下文，几轮之后模型就会”读不动、抓不准、判不出”。

讲者给出的方案是把上下文处理拆成显式的四步流水线，每一步只做一件事：

5.5.2 第 1 步——Raw CLI Output（原始命令行输出）

工具调用的原始结果，特点是长、噪声多、重复信息多、AI 不友好：典型的 `nmap` 输出夹杂大量 ANSI 控制符、`gobuster` 输出夹杂上千条 404、HTTP 响应里塞满 cookie 与冗余 header。这些内容如果直接进入上下文，会迅速逼近窗口上限。

5.5.3 第 2 步——RTK Rewrite（工具结果重写）

RTK Rewrite 是 Context Harness 的第一道净化器：把工具的原始输出按工具类型规范化为”AI-friendly 的命令式输出”，提取关键字段、丢弃噪音。讲者第 192–193 句用一句话概括了它的目标——”AI-friendly 的一个命令的友好输出”。

RTK Rewrite 的四类规范化对象：

- **Nmap 扫描结果**：去除 ANSI、保留开放端口/服务/版本三列；
- **HTTP 响应**：去掉重复 header，保留状态码、关键 header、可见 body 摘要；
- **目录扫描**：保留 200/302/403 等可疑状态码项，丢弃大批 404；
- **其它（JSON、文件列表等）**：按结构化层级整理为可被 Agent 直接消费的紧凑格式。

RTK 的英文落点是 Tool Result Rewrite Kit；字幕错听形态有”RT k rewrite / RTa rewrite / RT 重写”等。

5.5.4 第 3 步——Context Compression / Compaction（上下文压缩）

第三步处理的是”会话级”压力：当主上下文逼近窗口上限（演讲者多次提到的工程经验值是 80%–90% 阈值），Compaction 被触发。这一步内部有四个子产物：

- **Session Compaction**：长会话精简化，对历史轮次做摘要式重写；
- **Memory Summary**：把 Memory Board 的事实凝练成可直接注入的摘要；
- **Idea Summary**：把 Idea Board 的活跃假设凝练成摘要；
- **Query History On Demand**：超长工具结果不进入主上下文，而是文件落地保存，主上下文只保留摘要 + 文件路径，模型可”按需检索”原始内容。

最后一项是关键设计——它把”全部内容塞给模型”换成了”摘要 + 索引 + 检索”，让上下文从”线性堆积”变成了”分级访问”。

5.5.5 第 4 步——Solver Context（注入 Solver 的紧凑上下文）

经过前三步处理后，最终注入 Solver 的上下文只剩四块：Compacted Session、Memory Summary、Idea Summary、Query History When Needed。Solver 看到的不再是原始日志洪流，而是一份”已经被旁观者读过、整理过、归档过”的工作交接单。

5.5.6 Progressive Disclosure: Skill 系统的同源思想

讲者把这一整套流程命名为”渐进式披露（Progressive Disclosure）”，并明确类比 Claude Code 的 Skill 系统：“就实现了像 Skill 一样的渐进式披露。”两者底层是同一个观念：先给 Agent 摘要 + 入口，让 Agent 自己判断是否需要更深的内容、再去主动加载——而不是一次性把所有内容堆到上下文里”做最坏假设”。

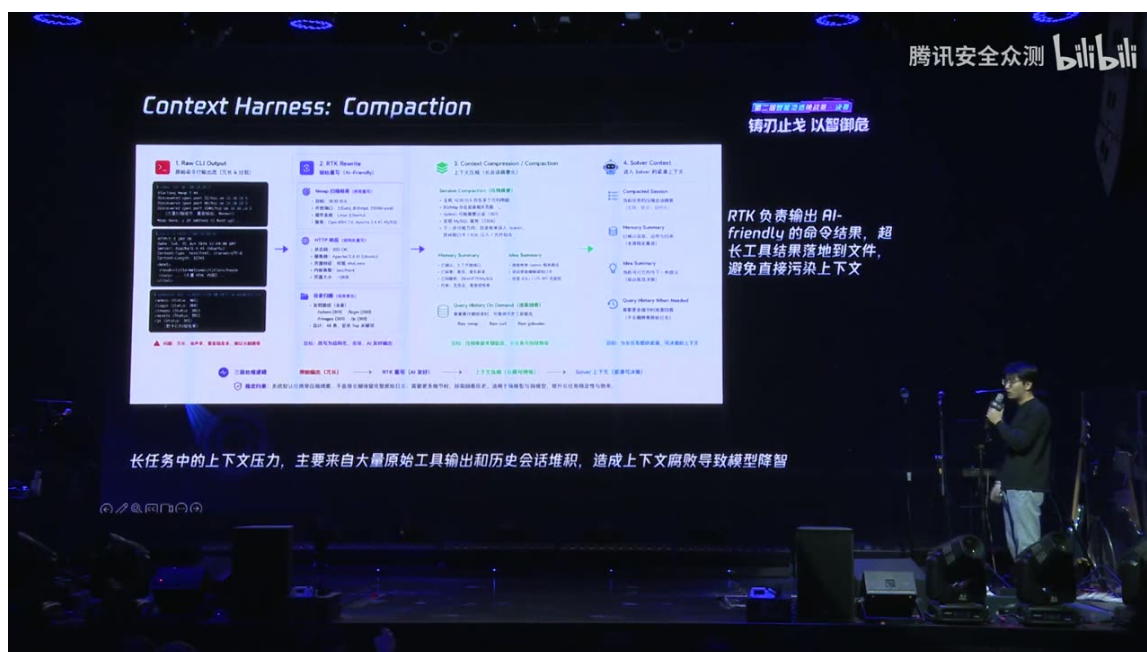


图 9: Context Harness: Compaction 四步流水线。第 1 列 Raw CLI Output（原始噪声大）→ 第 2 列 RTK Rewrite（AI-friendly 改写）→ 第 3 列 Context Compression（Session Compaction + Memory/Idea Summary + Query History On Demand）→ 第 4 列 Solver Context（紧凑可决策）。底部三层加速链清晰呈现”原始 → 改写 → 压缩 → 注入”。

4

5.6 本章小结：四大创新如何共同构成”稳定运行环境”

四大创新看似分散，实则一一对应 Solver 长程稳定性的四个失败模式：

- 方向漂移 → 智能纠偏（Observer 在外侧拉回主线）；
- 状态紊乱 → Memory & Idea 双层状态板（事实/假设分开归档）；

⁴视频画面时间区间：00:08:08（frame_0245）。

- 过早结束 → Ralph-Loop（外部条件取代主观判断）；
- 上下文腐败 → Compaction 四步流水线（噪音 → 摘要 → 索引 → 注入）。

这正是讲者在第 200–202 句的收口：“以上四点构成了我们智能渗透 Agent 的一个稳定运行的环境。”回到第 1 节的“四条设计理念”，这一节同时呼应了三条——“智能决策要有工程边界”（Ralph-Loop / RTK）、“经验要沉淀成共享状态”（Memory & Idea）、“架构要做稳定骨架”（整套 Harness 都是稳骨架而非加枷锁）。剩下的“执行环境必须隔离可重置”已在 §2 的 Kali 容器一节落地。

把四个支柱合在一起看，它们其实在做同一件事：把模型不可控的部分用 **Harness** 包住，留给 **LLM** 的只剩下高价值的判断力——这一句也将在 §6 的总结中被再次强调，作为整套 Adaptive Architecture 的工程灵魂。

6 实验数据、Q&A 与总结

讲者 B 在介绍完 Harness 四大创新之后，把演讲推进到第三段——“我们针对 Agent 的表现，从数据层面来分析一下”。这一段先用两张图展示比赛实测数据，随后进入三轮评委 Q&A：评委 1 关心独特优化是否独立于 Claude Code，评委 2 关心 Idea 维护稳定性与上下文压缩细节，评委 3 关心全局 Memory 是否会无限增长。在记录完三轮答辩之后，本节给出笔者对全篇的结构化提炼与延伸思考。

6.1 实验数据一：成本与一血/二血/三血分布

讲者第 205–216 句解释了“Cost”图表的语义。需要先纠正一个口语和图表的错位：

图表标题与口播略有出入。讲者口播说“蓝色坐标是执行时间、折线是 token 消耗”，但幻灯片实际标题是 **Cost**（横轴 = 赛题，纵轴 = 金额，单位人民币元换算的美元成本），并不是单纯的“性能曲线”。“一血/二血/三血”是图中通过红/橙/黄三色虚线方框标注的赛题分段，而不是单条曲线。本笔记按幻灯片含义解读图意。

按幻灯片可读出的关键信息：

- **总成本**：Total Cost = $\$2757.6251 \times 2.5 + 491.79 \text{ ¥(kimi)} \approx 7385 \text{ 元}$ ；
- **峰值赛题**：OauthLink 七色（约 2128.9）、Layer Breach（约 2061.8）、Final Challenge（4000+）等综合性赛题占用了主要预算；
- **一血/二血/三血分段**：橙/红/黄三色方框分别圈出“较早被夺一血”“重点突破时段”“后期收尾时段”对应的赛题群。

讲者引出这张图的真正目的不在数字本身，而是回到一个设计前提：**比赛是无人值守环境**（第 213–214 句：“比赛场景就是无人值守”）。无人值守意味着选手提交后不可干预，Agent 必须自主推进；这反过来逼出了整套架构的设计目标——**高效且稳定**（第 216 句）。

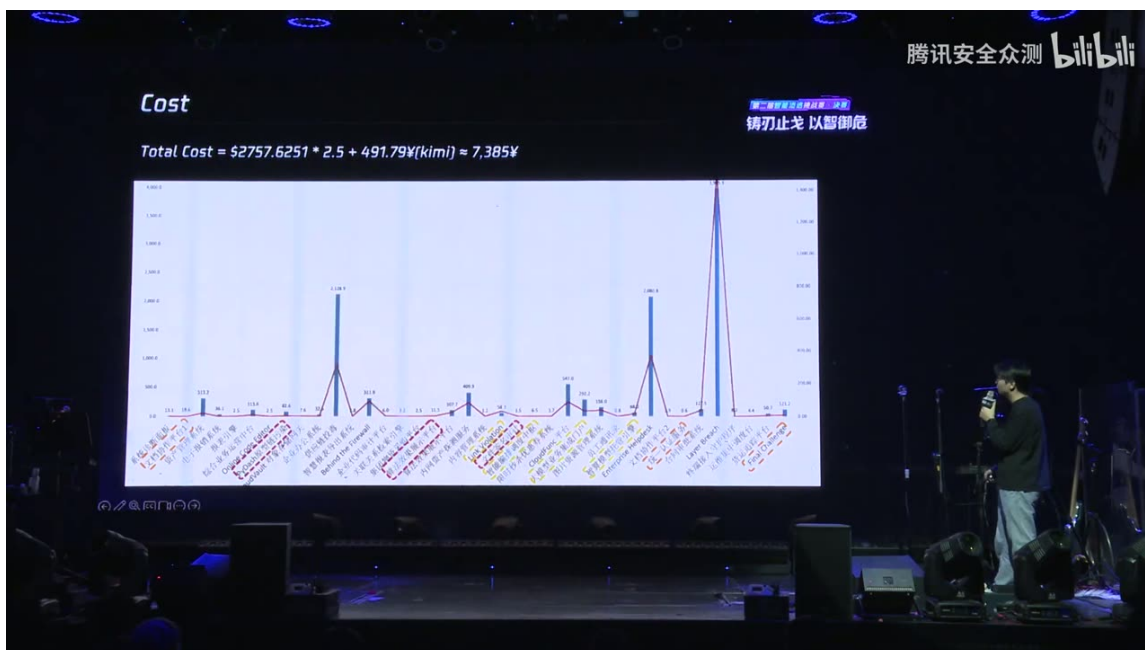


图 10: 比赛成本分布图（幻灯片标题”Cost”）：横轴为各赛题，纵轴为成本金额（美元 $\times 2.5$ + kimi 人民币）。蓝色柱为单题花费、红色折线为累计趋势；底部红/橙/黄三色虚线方框分别圈出一血、二血、三血赛题段。

5

6.2 实验数据二：海外 + 国产模型并行的 Model Performance

第二张图揭示了团队真正的”模型策略”——并不是依赖某一个超强模型，而是每道题用一个海外模型 + 一个国产模型并行探索（第 220–221 句）。表格分行列出了各模型的 Challenges / Flags / Flag Rate / Error / Time / Tokens / Quality:

- **claude-opus-4-6**（37 Challenges、42–53 Flags、约 79% Flag Rate、6713 min、1.76M tokens）——主力突破模型；
- **glm-5.1、kimi-k2.5、glm-5-turbo**——国产模型并行覆盖（kimi-k2.5 跑了 27 道题、3–27 Flag、约 11% Flag Rate）；
- **gpt-5.4、gpt-5.3-codex、gemini-3-pro-preview**——海外模型补充覆盖。

讲者在第 222–227 句给出了关键解读：”不是强调每个模型到底谁更强，而是在我们这个架构下，每个模型都有自己能竞争的地方，能共同覆盖更全的面与成功率。所以说在我们这工程架构下，国产模型仍具备一定的竞争力。”这一句话非常关键——它把”模型选择”从”单点比拼”重置为”组合策略”，并且把”国产模型仍具竞争力”的功劳归到 **Harness 工程架构**上而非模型本身。

⁵视频画面时间区间：00:08:44（frame_0263）。

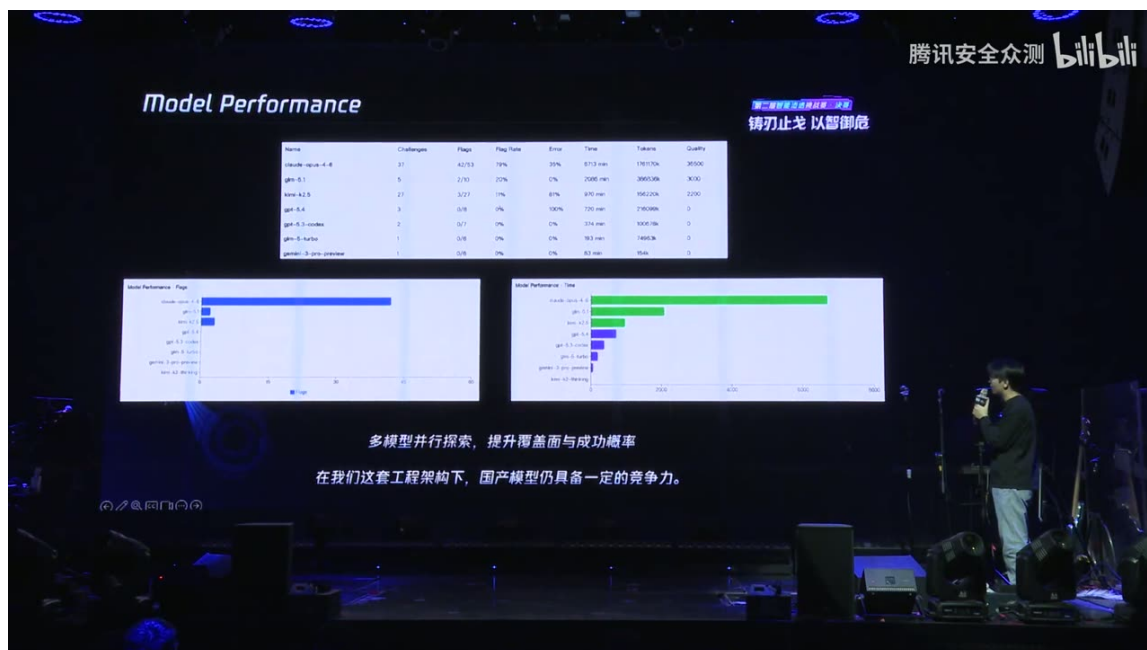


图 11: Model Performance: 上方表格列 Name / Challenges / Flags / Flag Rate / Error / Time / Tokens / Quality; 下方两幅水平柱状图——左 Flags (claude-opus-4-6 远超其他)、右 Time (claude-opus-4-6 / glm-5.1 / kimi-k2.5 占用时间最多); 底部金句”多模型并行探索，提升覆盖面与成功概率; 在我们这套工程架构下，国产模型仍具备一定的竞争力。”

6

6.3 Q&A 第一轮：与 Claude Code 范式的差异

6.3.1 Q1: 三个亮点要”不依赖模型，也不依赖 Claude Code”

评委 1 在第 232–241 句问得很尖锐：能不能讲三个亮点，证明这次的工作既不依赖于模型本身、也不依赖于 Claude Code 这种现成框架？

讲者答（第 242–248 句）：”我们不是不依赖模型，而是知道模型能力很强，也知道弱模型的差距，所以用工程优化尽可能弥补弱模型差距、放大强模型优势。”——这是一个非常诚实的回答，没有把”工程”和”模型”对立起来，而是把工程定位为”两端拉平器”：拉高弱模型下限、拉高强模型上限。这恰恰回应了 §5 的 Efficiency Reminder 与 Llama 短板的关系。

6.3.2 Q2: 推理框架是 Claude Code/Codex，还是自研？

评委 1 紧接着追问框架来源（第 249–250 句）。

讲者答（第 251–261 句）：”我们的推理框架用的 PydanticAI SDK，它其实就是 open Cloud 底层框架……整体实现非常优雅，扩展性也非常高，所以我们针对非常细致的地方做一些优化。”

⁶视频画面时间区间：00:09:18 (frame_0280)。

讲者口误澄清: PydanticAI $\neq$ Claude Code 底层框架。讲者把 PydanticAI 描述为”open Cloud (即 Claude Code) 的底层框架”, 这是一个明显的归属错误。事实上:

- **Claude Code** 的底层是 Anthropic 的 **Claude Agent SDK**;
- **PydanticAI** 是 Pydantic 团队独立维护的另一套 Agent SDK;
- 两者是**平行框架**, 思想上同根(都遵循”工具调用 + 状态机 + Harness”范式), 但代码上互不依赖。

读者在引用本演讲结论时务必区分。本笔记在涉及”框架选型”时一律以 **PydanticAI** 为准。

Claude Code 范式与 PydanticAI 的关系: ”Claude Code 范式”是评委用以泛指 Agent 工程参考系——以 Claude Code、Cursor、Codex 等为代表的”工具调用 + 长会话 + 上下文压缩”现代 Agent 工程实践。PydanticAI 与该范式属于**思想同根、实现独立**: 都拥抱 Skill / Tool 系统、Session Compaction、Workspace 文件落地等设计, 但在 SDK 层面是两套独立产品。本演讲团队的贡献是在 **PydanticAI** 上叠加一层 **Pentest 专用 Harness** (Observer Sidecar / Ralph-Loop / RTK Rewrite / Memory & Idea), 从而走出与 Claude Code 不同的差异化路线。

6.3.3 Q3: 能否量化证明优于 Claude Code?

评委 1 第三问 (第 262–269 句): 能否拿出量化指标证明”PydanticAI + Harness”比 Claude Code 更强?

讲者答 (第 270–275 句): ”通过这次比赛的成绩, 就可以看到我们的 PydanticAI 加我们的 Harness 这些优化插件, 是足以**超越其他队伍**的——这是我的底气。”——这是一个”以结果证明过程”的回答: 本届第 1 名 (且与上届第 7 名形成强烈对比) 就是工程价值最直接的量化证据。

6.4 Q&A 第二轮: Idea 维护与上下文压缩

6.4.1 Q: Idea 怎么保证不被改写、攻击方向不偏离?

评委 2 在第 277–286 句问了一个非常关键的问题: Idea 在多 Agent 协作中, 如何保证不被其它 Agent 误改、攻击方向不被搞偏?

讲者答 (第 287–303 句): 用一句话概括是 ”**两层 Idea/Memory + Observer 单点维护权**”:

- **两层架构**: Challenge 集 (全局) 一层 + 每个 Solver 内部 (局部) 一层;
- **Solver 不能写 Memory/Idea**: Solver 局部状态只能由它配套的 **Observer Sidecar** 动态维护;
- **假设–验证–证实–沉淀的环路**: Observer 提出假设 (Idea: Pending) $\rightarrow$ 主 Solver 执行验证 (Idea: Testing) $\rightarrow$ Observer 观察执行结果 $\rightarrow$ 假设证实后才沉淀为 Memory (Idea: Verified) $\rightarrow$ 作为下一次突破的起点。

这个回答与 §5 的 Idea Board 状态机完全对齐：改写 Idea 是 Observer 的特权，不是 Solver 的权利。Solver 可以读，但写入 / 合并 / 删除统一交给 Observer。这就是评委关心的”攻击方向不被偏离”的工程保障。

6.4.2 Q：上下文压缩具体怎么做？

评委 2 的第二问（第 305–308 句）请讲者展开”上下文压缩”。这是讲者在 §5 第四点的延伸，但答得更具体——分成了运行时（in-loop）和会话级（session-level）两层。

运行时层（讲者第 309–325 句）：基于 PydanticAI 插件 + RTK Rewrite + 工具结果落文件，主上下文只保留摘要 + 文件路径，Agent 按需检索；类比 Skill 系统，实现”渐进式披露”。

会话级层（第 326–349 句，由另一位讲者补充）：当主上下文达到 80%–90% 阈值时触发 Session Compaction（即”压缩”动作）。但讲者在这里强调了一个非常关键的设计：

80%–90% 阈值与 Workspace 保留模式：

- **压缩触发线：**主上下文使用率达到 80%–90%；
- **原始 Session 不删除：**完整历史会话被搬移到 Agent 的 **Workspace** 目录，作为外存保留；
- **Prompt 提示检索方式：**在压缩后给 Agent 补一句”你的原始 session 在这里，可以 **grep / glob** 去检索原始内容”；
- **效果：**精度损失几乎可以不计，整体流程稳定轮转。

注：讲者口误把”glob/blob”说成了”blow”，幻灯片含义为”按文本/模式检索原始 session 文件”。

这个设计是整套架构里最具迁移价值的一招——**压缩不是丢弃**。它把一次性裁剪上下文的”硬丢失”，换成了”主上下文做摘要 + Workspace 留全本 + 模型可主动 grep”的三段式结构，相当于让 Agent 拥有了”短期记忆 + 可检索长期记忆”的双层结构。

6.5 Q&A 第三轮：全局 Memory 增长可控的三道闸门

评委 3 在第 352–360 句把上届选手暴露的问题直接抛给了本届团队：你们的全局 Memory 是否也会”一直增加不删减、最终上下文越来越长”？

讲者答（第 361–386 句）给出了一个非常体系化的回答——Memory 不是无限增长的，因为有三道闸门共同保证它的可控状态：

Memory 增长可控的三道闸门：

1. **Solver 只读**——Solver 不能 update / delete Memory，记忆不会被多个 Solver 同时乱写；
2. **Observer 维护权**——Memory 的整理 / 合并 / 删除必须经过 **Observer Agent**，由它判断“哪些当前阶段重要、哪些不重要”，重要的增强、不重要的删除；
3. **Prompt 阈值约束**——在 Observer 的提示词里写明“记忆不是越多越好，只有事实假设级别的、非常确定的东西才允许写入 Memory / Idea”。

三道闸门叠加：写入有门槛、维护有专人、提示有约束——这才是“可控记忆”的工程保证。

讲者在这段答辩中还顺带回答了“Solver 是否会因为 Memory 变化而困惑”的问题——**Observer 改变记忆后会通知 Solver**（第 369-371 句）：“observer 改变了记忆，他会通知到 Solver，告诉他我现在有哪些东西可以给你看。”因此 Solver 的工作循环里只有“做事 + 接受推送”，并不需要关心记忆怎么管理。

至此三轮 Q&A 圆满收官。讲者第 228-229 句的“以上就是 AI 小分队的分享”是演讲主体的收口；第 304 / 350 / 351 句的“谢谢”则各自标记三轮答辩的结束。

6.6 总结与延伸

6.6.1 6.1 团队主张的核心贡献

剥去比赛胜负的修辞，这次演讲真正交付的是一套 **Adaptive、可配置、可复盘的 Pentest Agent Harness**：

- **Adaptive**：Manager 调度感知战场状态、Solver 分裂 + 合并、Observer 动态维护记忆——三个层面都是动态而非静态；
- **可配置**：策略 / 模型 / Prompt / 工具 / Skills / MCP 都可页面动态调整（§2 工程实践视图）；
- **可复盘**：Solver 日志、Memory、Idea、提交记录全量留痕，从拿 Flag 回溯到路线、成本、关键决策。

6.6.2 6.2 与 Claude Code 范式的关系

讲者在 Q&A 第一轮反复用“open Cloud 底层框架”来描述 PydanticAI——技术口径不准，但精神归属是对的：本架构与 Claude Code / Cursor 等 **Claude Code 范式同根（PydanticAI / Claude Agent SDK 风格）**，但在垂直渗透场景下做了五项关键加法：

- **Observer Sidecar**（每个 Solver 配一只裁判）；
- **Ralph-Loop**（终止判断从模型主观抽离到系统状态）；
- **RTK Rewrite**（工具结果改写为 AI-friendly）；
- **Memory & Idea 双层状态板**（事实 vs 假设分开归档）；
- **并行 Solver 广播**（Observer 汇总后写回全局共享态）。

这五项加法是把“通用 Agent SDK”垂直化为“Pentest Agent Harness”的核心增量。

6.6.3 6.3 笔者的结构化提炼

把整篇讲座的工程价值提炼成一句话：

Adaptive Architecture 的工程价值核心——

把模型不可控的部分用 Harness 包住，留给 LLM 的只剩下高价值的判断力。

”模型不可控的部分”包括：终止判断（漂移与早停）、注意力近端偏置（漂移）、工具结果污染（上下文腐败）、多轮记忆膨胀。这些都用工​​程手段——状态机、外部约束、文件落地、专人维护——固化下来；而 LLM 只在”如何利用 Memory & Idea 做下一步判断”这件高价值的事上投入算力。

把这一观点投影到全篇六节，可以看到一条清晰的”工程边界”主线——

- **Manager 工程边界** (§3)：把 LLM 的调度决策框在 Manager Tools 八件套和 Strategy Actions 六件套里；
- **Solver 经验沉淀** (§4)：把多 Solver 的局部探索经过 Observer 过滤后再广播；
- **Observer 智能纠偏** (§5)：把”是否漂移”的判断从主 Solver 卸载到 Observer；
- **Ralph-Loop 终止约束** (§5)：把”是否结束”的判断从模型主观转移到系统状态；
- **RTK + Compaction 上下文压缩** (§5)：把”读什么”用四步流水线 and Workspace 模式做工程接管；
- **三道 Memory 闸门** (§6)：把”记忆怎么涨”用 Solver 只读 + Observer 维护 + Prompt 阈值三层冗余兜住。

每一处都是同一个动作：把模型主观最不可靠的环节，用工程手段替换为可观测、可约束的状态。

6.6.4 6.4 未来工作与开放问题

跳出本届比赛的语境，这套架构未来还有几个值得展开的方向：

- **模型组合策略的进一步优化**：本届的”海外 + 国产并行”是粗粒度的随机分配，未来可以根据题型特征（注入 / 逆向 / 协议 / 综合）做**模型路由**（model routing），让每道题匹配最强的模型组合；
- **跨任务长期记忆**：当前 Memory 是 Challenge 级（赛题级），如果把”对某类漏洞的成功路径”沉淀为跨赛题/跨比赛的长期 Memory，整支队伍的能力会随时间复利；
- **Observer 自我迭代**：Observer 当前是”规则 + LLM”的混合体，未来如果让 Observer 自身的纠偏规则从历史 session 中自学习，可能形成”Observer 训练 Observer”的递归改进；
- **Ralph-Loop 在非 CTF 真实场景的迁移**：CTF 场景中”完成”的定义是 Flag，工程上很容易判定；真实渗透 / 蓝军任务里”完成”是更模糊的目标，Ralph-Loop 的”系统状态约束”如何泛化是一道开放题；

- **Skills 与 MCP 服务的进一步标准化**：当前 Skills 列表（init / anet-pentest / agent-browse / payloads-all-the-things 等）是团队私有，未来沿 **MCP** 协议标准化为可被多框架（Claude Code / Cursor / PydanticAI）共用的工具服务，迁移价值会数倍放大。

6.6.5 6.5 一句话收尾

如果一定要给整套 Adaptive Architecture 一个收尾的注脚——

”每一次探索都不应消失，而应沉淀为下一次突破的起点。”

——这是 Solver 设计第一性原则；放大到整支队伍，这也是 Adaptive Architecture 的整套哲学：模型可以换、赛题可以变，但 **Harness** 沉淀下来的工程经验，会成为下一届、下一个真实战场上、下一支团队的起点。



图 12: 致谢页“THANKS”——绿盟科技 AI 小分队两位讲者并肩站台，背景为腾讯云安全 / TCH 腾讯云黑客松 / 腾讯安全众测三方 logo 与“铸刃止戈以智御危”赛事标识。

7

⁷视频画面时间区间：00:09:54（frame_0298）。